

CPSC 304 Project Cover Page

Milestone #: 3

Date: March 6, 2026

Group Number: 107

Name	Student Number	Alias (Userid)	Preferred E-mail Address
Justin Hoang	94126356	t3v8j	Just.hoang@outlook.com
Hao Hu	12260022	t9s7z	helenhaohu@gmail.com
Angel Shen	98425549	i0n5o	an.shen222@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

University of British Columbia, Vancouver

Department of Computer Science

2. Summary:

Our project allows users to upload videos into a database and assign words or phrases to certain movements, which are used to calibrate a machine learning model. The user can then record or upload videos into the program and receive a transcript of the phrases or words detected from the video by the model.

3. Timeline and Tasks:

Dates	Tasks
March 7 - 9	- Debug SQL DDL with the rest of the demo program (make sure the demo program runs with our current SQL implementation) - Edit existing SQL to include more advanced SQL queries (UNION, INTERSECT, GROUP BY... etc) for Milestone 4 - ensure that all 10 queries are different and no query can be a subset of the other** - INSERT query (Justin) - UPDATE query (Hao) - DELETE queries (Angel) - Possibly remove/optimize current number of tables (Angel) - Begin implementing user login process - link user database to store user details (Justin) - create text for user creation process and test user database insertion from UI (Angel) - create basic placeholder icons and buttons for user page (Hao)
March 10 - 17	- Continue working on SQL for milestone 4 - SELECTION query (Justin) - PROJECTION (Angel) - JOIN query (Hao) - GROUP BY query (Justin) - HAVING query (Angel) - nested GROUP BY query (Hao) - DIVISION query (Justin) - Tune/optimize hyperparameters and decide on the final machine learning model for the program (Justin) - Connect OpenCV to our database so that it can analyze existing videos (Hao) - Test that recorded videos are being parsed and uploaded to our database properly (Hao) - Train and test machine learning on live and recorded footage - basic, easy to read gestures with a plain background (easily interpretable footage) (Angel) - compare machine's transcript with expected transcript (Angel)
March 18 - 25	- Continue working on user login process - test data persistence for users (ie. user uploaded videos remain with that user even after closing the program) (Hao) - Ensure user has the ability to delete recordings and transcripts - link buttons from UI to the database (Angel) - Continue testing machine learning on live and recorded footage

University of British Columbia, Vancouver
Department of Computer Science

	- test on harder to read footage ie blurrier videos with lower resolution, colourful backgrounds, multiple moving parts in the background (Justin) - If machine learning model is not predicting very well (machine accuracy is low) re-tune hyperparameters and reevaluate model choice (Angel)
March 25 - April 2	- Create dashboard UI using React - plan UI design/layout (Hao) - implement buttons and link to functions (Justin) - create proper icons (Hao) - implement on-click refresh (Angel) - test UI is properly inserting data into the database (Hao) - Create website navigation bar (Angel) - refreshes page on click - Create UI for transcript - ability to edit font size (Justin) - button to retrieve transcript from database for download (Justin)
April 2 - 4	- Final bug fixes to ensure the project can be run easily from the project environment (Justin) - Final testing to ensure the machine learning model is interpreting gestures correctly (Angel) - Milestone 4 final edits (Hao)

4. Milestones 1 and 2 have both been added to the repository.

5. DDL:

The DDL statements from the previous milestone and all the insert statements have been added to the file **setup.sql**.

6-9: Not applicable for this PDF document

10. AI-Declaration:

Due to the machine learning and motion sensor components of our project, Python was the most practical choice for the backend. Most of our experience with machine learning libraries comes from Python, particularly from CPSC 330. We also have little knowledge of coding machine learning models from scratch otherwise, so Python felt like the only real choice. However, none of our team members had experience connecting a Node.js and JavaScript project to a Python backend. To help with the initial setup, **WE USED AI ASSISTANCE** (ChatGPT) to help guide our project structure and properly connect Python, React, FastAPI, and the Oracle database.

This is a link to a pdf of the full conversation:

<https://drive.google.com/file/d/1Ds6iV9m3b7UdvfeMEFbSJ1IUVSAdA9yr/view?usp=sharing>

The following pages also include screenshots of the full ChatGPT conversation.

ChatGPT  Free offer 

Right now I have a project that uses javascript and node.js as a frontend. I want to use python with FastAPI and react. Can you give advice on how I should complete the transition? I want a backend in python with FastAPI and a frontend with react. For now I want a simple vite + react frontend.

Transitioning from a **Node.js + JavaScript frontend** to a **Python FastAPI backend + React frontend (Vite)** is a very common modernization path. The cleanest way is to **separate backend and frontend into two independent apps** that communicate via an API. Below is a practical transition plan that minimizes breakage and lets you migrate gradually. 🚀

1. Target Architecture

Your final structure should look like this:

```
project-root/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── routers/
│   │   ├── models/
│   │   └── services/
│   ├── requirements.txt
│   └── venv/
├── frontend/
│   ├── index.html
│   ├── package.json
│   ├── vite.config.js
│   └── src/
│       ├── App.jsx
│       ├── main.jsx
│       └── components/
```

3/6/26, 9:47 PM

React FastAPI Transition Guide

```
|  
└─ docker-compose.yml (optional)
```

Responsibilities

Layer	Technology	Responsibility
Backend	FastAPI	API + business logic
Frontend	React (Vite)	UI
Communication	REST / JSON	fetch / axios

2. Create the FastAPI Backend

First build the backend **independently**.

Install dependencies

```
</> Bash  
  
cd backend  
  
python -m venv venv  
source venv/bin/activate # linux/mac  
venv\Scripts\activate    # windows  
  
pip install fastapi uvicorn python-multipart
```

Create:

```
backend/app/main.py
```

```
</> Python  
  
from fastapi import FastAPI  
from fastapi.middleware.cors import CORSMiddleware  
  
app = FastAPI()  
  
origins = [  
    "http://localhost:5173",
```

3/6/26, 9:47 PM

React FastAPI Transition Guide

```
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/")
def root():
    return {"message": "FastAPI backend running"}

@app.get("/api/data")
def get_data():
    return {"data": ["apple", "banana", "orange"]}
```

Run backend:

```
</> Bash

uvicorn app.main:app --reload
```

Server:

```
http://localhost:8000
```

Docs automatically available:

```
http://localhost:8000/docs
```

3. Create the React + Vite Frontend

From the root:

```
</> Bash

npm create vite@latest frontend
```

Choose:

3/6/26, 9:47 PM

React FastAPI Transition Guide

React
JavaScript



Install dependencies:

```
</> Bash  
  
cd frontend  
npm install
```



Run:

```
</> Bash  
  
npm run dev
```



Frontend:

<http://localhost:5173>



4. Connect React to FastAPI

Edit:

frontend/src/App.jsx



```
</> JavaScript  
  
import { useEffect, useState } from "react"  
  
function App() {  
  const [data, setData] = useState([])  
  
  useEffect(() => {  
    fetch("http://localhost:8000/api/data")  
      .then(res => res.json())  
      .then(data => setData(data.data))  
  }, [])  
  
  return (  
    <>
```



3/6/26, 9:47 PM

React FastAPI Transition Guide

```
<h1>FastAPI + React</h1>

<ul>
  {data.map((item, i) => (
    <li key={i}>{item}</li>
  ))}
</ul>
</>
)
}

export default App
```

What is this? ⓘ

Now React fetches data from FastAPI. 🇺🇸

5. Recommended FastAPI Project Structure

As your backend grows, use this structure:

```
backend/app/

main.py
dependencies.py

routers/
  users.py
  auth.py
  stocks.py

models/
  user.py
  stock.py

schemas/
  user.py
  stock.py

services/
  user_service.py
  stock_service.py

database.py
config.py
```


Example router:

 Python



 Run

```
from fastapi import APIRouter

router = APIRouter(prefix="/users")

@router.get("/")
def get_users():
    return ["user1", "user2"]
```

6. Handling Environment Variables

Frontend:

frontend/.env



VITE_API_URL=http://localhost:8000



Use it:

 JavaScript



```
fetch(`${import.meta.env.VITE_API_URL}/api/data`)
```

7. Running Both Together

Two terminals:

Terminal 1

```
cd backend
uvicorn app.main:app --reload
```



Terminal 2

```
cd frontend  
npm run dev
```



8. Optional: Use Docker Compose

Since you already use **Docker Compose** in your RankAlpha project, you could do:

```
services:  
  
  backend:  
    build: ./backend  
    ports:  
      - "8000:8000"  
  
  frontend:  
    build: ./frontend  
    ports:  
      - "5173:5173"
```



9. Migration Strategy (Important)

Instead of rewriting everything at once:

Step 1

Keep Node backend running.

Step 2

Start rewriting endpoints in FastAPI.

Step 3

Point React to FastAPI endpoints.

Step 4

Delete Node backend.

This avoids breaking your frontend.

10. Tools I Strongly Recommend

FastAPI ecosystem:

```
pydantic  
sqlalchemy  
alembic  
python-dotenv  
httpx
```



React ecosystem:

```
axios  
react-router-dom  
tanstack-query
```



11. The Biggest Mistake to Avoid ⚠️

Do NOT try to serve React from FastAPI during development.

Use:

```
React dev server (Vite)  
+  
FastAPI API server
```



Production bundling comes later.

12. If You Want, I Can Also Show

- ★ A complete FastAPI + React starter template
- ★ A recommended folder structure for large projects
- ★ How to add authentication (JWT)
- ★ How to connect PostgreSQL with FastAPI

Just tell me and I can generate a clean production-ready template.

can you give a very basic folder structure and template,
just so I can get started

Here is a **very minimal FastAPI + Vite + React template** you can start with. The goal is **simplicity**, so you can get everything running in a few minutes and expand later. 🚀

Project Folder Structure

```
</> Plain text
my-project/
├── backend/
│   ├── main.py
│   └── requirements.txt
├── frontend/
│   ├── index.html
│   ├── package.json
│   ├── vite.config.js
│   └── src/
│       ├── main.jsx
│       └── App.jsx
└── README.md
```

Two completely separate apps:

- **backend** → FastAPI
- **frontend** → React (Vite)

They communicate through **HTTP requests**.

1 Backend (FastAPI)

`backend/requirements.txt`

University of British Columbia, Vancouver

Department of Computer Science

3/6/26, 9:47 PM

React FastAPI Transition Guide

</> Plain text



```
fastapi
uvicorn
```

Install:

</> Bash



```
cd backend
pip install -r requirements.txt
```

backend/main.py

</> Python



▶ Run

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# allow React dev server
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/")
def root():
    return {"message": "FastAPI running"}

@app.get("/api/hello")
def hello():
    return {"message": "Hello from FastAPI"}
```

Run the backend:

</> Bash



```
uvicorn main:app --reload
```

Backend runs at:

`http://localhost:8000`



Docs automatically available:

`http://localhost:8000/docs`



2 Frontend (React + Vite)

Create the frontend:

```
</> Bash  
npm create vite@latest frontend
```



Choose:

```
React  
JavaScript
```



Then:

```
</> Bash  
cd frontend  
npm install
```



frontend/src/main.jsx

```
</> JavaScript  
  
import React from "react"  
import ReactDOM from "react-dom/client"  
import App from "./App"  
  
ReactDOM.createRoot(document.getElementById("root")).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
)
```



```
</React.StrictMode>  
)
```

frontend/src/App.jsx

```
</> JavaScript  
  
import { useEffect, useState } from "react"  
  
function App() {  
  const [message, setMessage] = useState("")  
  
  useEffect(() => {  
    fetch("http://localhost:8000/api/hello")  
      .then(res => res.json())  
      .then(data => setMessage(data.message))  
  }, [])  
  
  return (  
    <div>  
      <h1>React + FastAPI</h1>  
      <p>{message}</p>  
    </div>  
  )  
}  
  
export default App
```



What is this? ⓘ

Run frontend:

```
</> Bash  
  
npm run dev
```



Frontend runs at:

```
http://localhost:5173
```



3 Running the Project

Open two terminals.

Terminal 1

```
</> Bash
cd backend
uvicorn main:app --reload
```



Terminal 2

```
</> Bash
cd frontend
npm run dev
```



Open:

```
http://localhost:5173
```



You should see:

```
React + FastAPI
Hello from FastAPI
```



4 What This Gives You

- ✓ Working React frontend
- ✓ Working FastAPI backend
- ✓ API communication
- ✓ CORS configured
- ✓ Minimal structure

5 Next Steps (Recommended)

Once this works, gradually add:

Backend

3/6/26, 9:47 PM

React FastAPI Transition Guide

```
backend/  
  main.py  
  routers/  
  models/  
  schemas/  
  services/
```



Frontend

```
frontend/src/  
  components/  
  pages/  
  api/
```



✅ If you'd like, I can also show you a **"clean beginner production structure"** that most FastAPI + React projects use (about **10 files total**) that scales much better once the project grows.